Control de la tira RGB IOTBT701 desde un portátil Windows: Ambilight por BLE

Informe analítico, 29 oct – 2 nov 2026 (4 sesiones) • github.com/Ne-collab228/iotbt701-ambilight • GUI v1.5

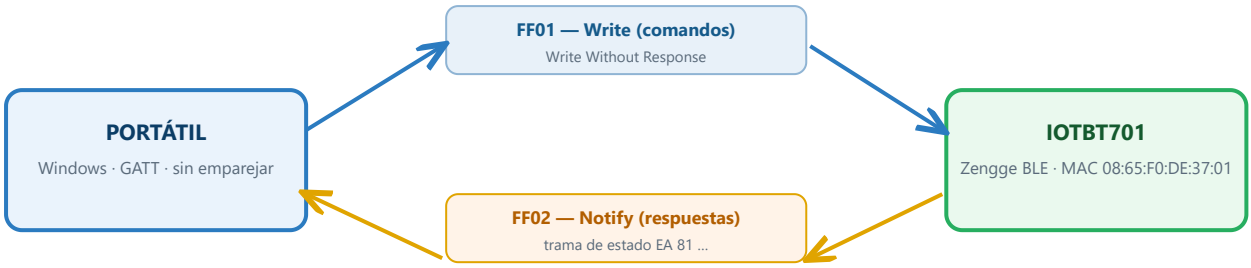
v1.5 GUI + arranque con Windows + fichero de registro + ventana «LINK LOST»	**10** commits, MIT, repo público	**5** idiomas de la documentación (EN/RU/ES/ZH/PT-BR)	**16.9/s** ritmo sostenido de escrituras (soak de 24,4 min)	**3** «falsos límites» eliminados por medición

Objetivo: la tira RGB **IOTBT701** (controlador Zengge/LEDnetWF por BLE, MAC 08:65:F0:DE:37:01) refleja el color medio de la pantalla del portátil. Sin app del teléfono, sin emparejamiento, Win32/.NET puro. Resultado: un producto GUI terminado, un repositorio público con documentación completa del protocolo y un conjunto de herramientas CLI — además, cada «límite» que al principio nos habíamos impuesto fue probado y eliminado honestamente.

1. Cronología

Fecha	Hito	Qué ocurrió
29–30 oct	**Protocolo descifrado**	Escaneo BLE sin emparejar (PairAsync de Windows siempre devuelve «Failed» — normal en este dispositivo, no es error). GATT directo, sesión con MaintainConnection. Se decodificó la envoltura de transporte: banderas/contador/fragmentos/delimitador 0x0B/checksum = sum(payload)&0xFF. Encendido/apagado verificados byte a byte contra las referencias de la integración lednetwf_ble (Home Assistant).
30 oct	**Color**	Las plantillas RGB antiguas (31/41/42/46) se ignoran: la familia IOTBT necesita el formato v3 `E0 01 00 A1` + tono/sat/brillo. 12 tonos seguidos ✓. Descubrimiento: los grados de tono impares los corrige mal el firmware → cuantizar a grados pares (error de 1°, invisible).
30 oct	**Ambilight v1.0**	AmbilightOT.exe: captura de pantalla, media ponderada del color, suavizado EMA, escenas oscuras → blanco, reconexión automática.
31 oct	**Publicación**	GitHub: docs PROTOCOL/AMBILIGHT/BUILDING, README ×5, GUI con selector de idioma en la propia ventana (52ea6c8), v1.1 — corrección de blanco cálido (el «blanco» de fábrica tira a azul), v1.2 — autoguardado de ajustes + curva de saturación contra el «centro deslavado» de los LED baratos.
1 nov	**v1.3 — retroalimentación**	¡La tira por fin RESPONDIÓ!: canal de consulta (familia 0x0A, sin checksum) en FF01 → trama de estado `EA 81 … F0 hue sat bri` en FF02. lednotify.exe. Límite documentado con honestidad: los registros de la tira solo los escribe la app del teléfono. Brillo: la escalera 100→130→255 probó «el byte es porcentaje 0..100»; >100 corrompe el color.
1 nov	**v1.4 — 20 Hz**	Se quitó nuestro propio tope de «≥350 ms»: sostenido de 16,9 cmd/s a 50 ms, cola-ráfaga de 456/s sin pérdidas, soak 3600/3600 en 213 s. GUI: el campo «intervalo» controla todo (muestreo de pantalla = techo de escritura, mín. 50 ms).
2 nov	**v1.5 + caza del bug**	Casilla «iniciar con Windows» (fuente de verdad: el acceso directo en shell:startup), ambilight_log.txt con precisión de ms, botón «registro de conexión» y ventana automática «LINK LOST» al desconectarse sola. En paralelo el usuario informa de una congelación de la tira a 50 ms tras 15–30 min — en investigación (§7).

2. Mapa de servicios BLE de la tira



Servicio extra FE00: FF11 (escritura) · FF22 (notify) — canal secundario del mismo protocolo, no necesario

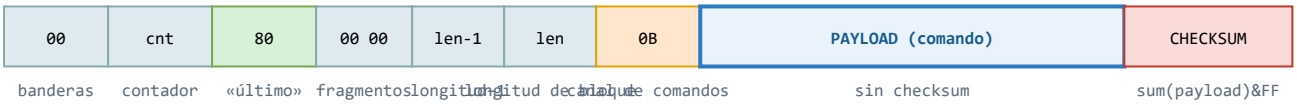
Algoritmo de conexión (arranque y reconexión)

```
1. BluetoothLEDevice.FromBluetoothAddressAsync(0x0865F0DE3701)   hasta 6 intentos × 5 s
2. GattSession.FromDeviceIdAsync(...) → session.MaintainConnection = true
   esperar ConnectionStatus == Connected                          hasta 6 intentos × 2 s
3. GetGattServicesAsync() → FFFF: FF01 (escribir) / FF02 (escuchar)
4. Escritura: FF01, WriteWithoutResponse; cada comando se pinta al instante
5. Al perder el enlace → ciclo completo de reconexión cada ~6 s (infinito)
```

> **Por qué sin emparejamiento:** PairAsync de Windows siempre falla con esta tira — pero no hace falta: las características GATT están abiertas. El emparejamiento solo bloquea el acceso.

3. La gramática del protocolo (todo verificado en la tira real)

3.1 Envoltura de transporte: cada byte en su sitio



Fórmula del checksum descifrada contra las referencias lednetwf_ble: suma de TODOS los bytes del payload desde 0x0B inclusive, módulo 256. Verificada byte a byte.

3.2 Comando de color (v3): 23 bytes que pintan el color

```
00 XX 80 00 00 0F 10 0B | E0 01 00 A1 HH SS BB 00 00 00 00 14 00 00 | CS
                        |   familia IOTBT      hue sat% bri%       |  CS = sum & 0xFF
HH = (hue << 7 | sat) >> 8 == hue/2   ·  SS = sat % (0..100)  ·  BB = bri % (0..100)
```

Comando	Payload tras 0x0B	Estado
Encendido / apagado	`3B 23/24 00…32 00 00`	✓ verificado
Color (v3)	`E0 01 00 A1 HH SS BB …`	✓ 12 tonos
Blanco	`3B A1 00 00 64 …` (sat=0)	✓
Plantillas RGB 31/41/42/46	`31 R G B …`	✗ la tira las ignora
Grado de tono impar	byte_lo = sat+128	✗ el firmware lo corrompe → cuantizar a par

bri > 100	escala porcentual	✗ corrompe el color → techo 100
Consulta de estado	`0B 81 8A 8B 96` (¡SIN checksum!)	✓ v1.3
Respuesta de estado (FF02)	`EA 81 01 00 AD 0A 23 25 6E 37 F0 hue…`	✓ decodificada

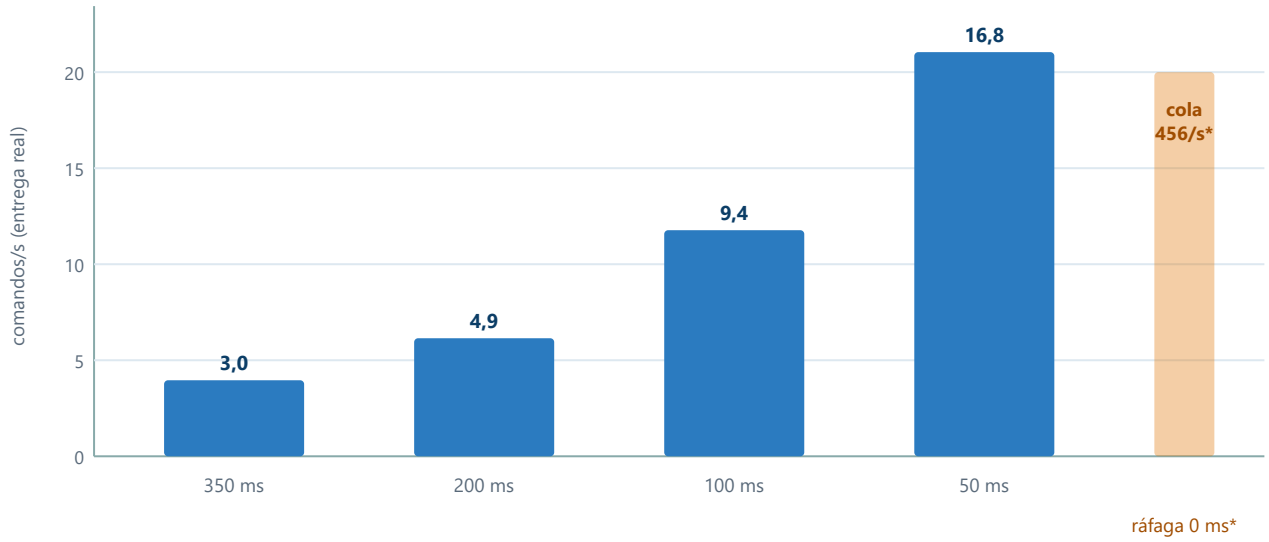
3.3 Por qué las plantillas «31 R G B» fallaban: esta tira es de otra familia

De QUICK_REFERENCE: `product_id = 0x00` (IOTBT/Telink) no es un solo tipo de dispositivo — **tres familias de comandos distintas** comparten ese id. Nuestra tira es la tercera familia («envoltura v3»: E0 01 00 A1). Las plantillas antiguas pertenecen a otras familias — un idioma extranjero para este aparato.

Familia	Enc/Apag	Color	Efectos	Ejemplos
Telink	0x71	`E2 0B hue lvl`	E0 02	IOTBT812
Segment	0x3B	`E1 03 …`	E1 01	IOTBT6BA
envoltura v3 (nuestra)	0x3B	`E0 01 00 A1 …`	—	ZJ-BBLA (PR #101)

4. Gráficos de rendimiento

4.1 Ritmo real de entrega con distintos intervalos



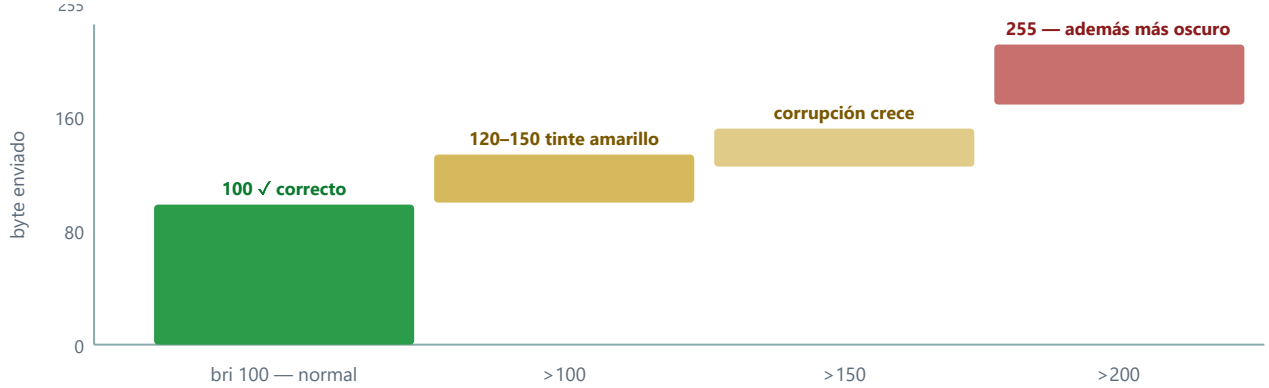
*Ráfaga = 100 comandos sin pausas: Windows los encola sin pérdidas, pero la radio solo transporta ~17/s — ese es el límite físico de la conexión. Por encima de 17 Hz nada queda más suave, solo más tarde.

4.2 Pruebas de resistencia (ambos pases: 3 min y 24 min a 50 ms)

Pase	Comandos	Entregados	Fallos	Ritmo	Duración	Hueco máx.
1 nov · soak #1	3600	3600/3600	0	16,9/s	213 s	141 ms
2 nov · soak #2	24000	24000/24000	0	16,4/s	24,4 min	125 ms

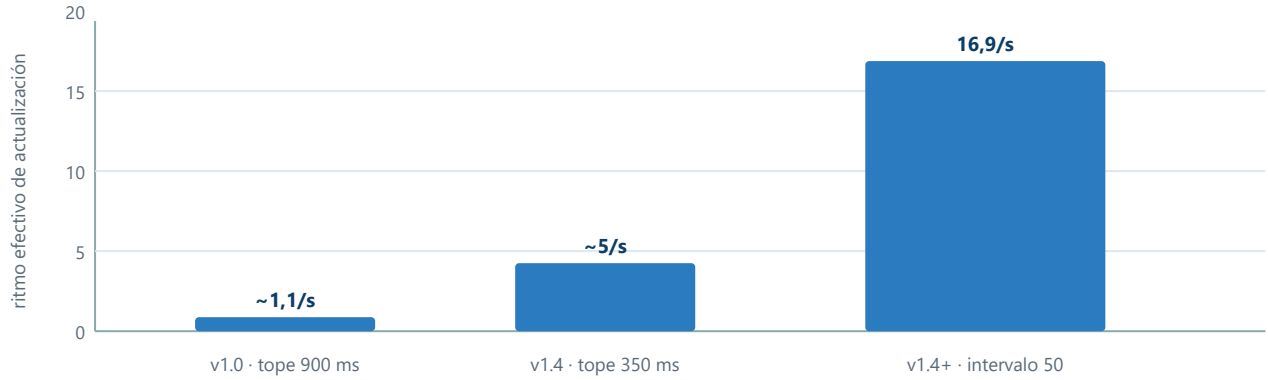
El ritmo nunca se degrada → el firmware no sufre bajo carga continua de 20 Hz. Tras cada pase: consulta de control + restauración del naranja: éxito.

4.3 Escalera de brillo (el fallo del firmware por encima de 100)



El byte bri es un PORCENTAJE 0..100. Los valores >100 se aceptan pero corrompen el color (tinte amarillo-verdoso; hacia 200–255 además se oscurece) — un fallo del firmware, no física de los LED.

4.4 Evolución: los topes que nosotros mismos pusimos, y cómo se retiraron



Tres «límites» no eran física sino exceso de cautela: «1–3 cmd/s», «hace falta hueco de 350 ms», «el brillo es 0–255». Retirados por medición: 20 Hz seguro, techo 100.

5. Herramientas

Herramienta	Propósito
AmbilightOT.exe (GUI)	El buque insignia: ajustes (intervalo 50 ms…3 s, ganancias sat/bri, brillo mín., umbral de blanco, corrección cálida), 5 idiomas de interfaz, autoguardado, reconexión automática; v1.5 añade la casilla «iniciar con Windows» (acceso directo en shell:startup), fichero de registro y ventana automática «LINK LOST».
ledctl.exe	Consola: white/on/off/rgb/hsv/sweep2 — control manual del color.
lednotify.exe	Retroalimentación y experimentos: query/listen/ladder/rate/soak/send.
blescan.exe	Escaneo BLE de 25 s — comprueba «¿está viva la tira en el aire» (cuando NADIE deja el enlace).
build_*.ps1	Compilación con csc.exe + fachadas GAC + WinMD — sin Visual Studio.

6. Lecciones técnicas (lo que no está en la documentación oficial)

- **Emparejar es dañino e innecesario**: PairAsync de Windows siempre falla en esta familia, pero las características GATT están abiertas.
- **El teléfono es monopolista del enlace**: con Surplife conectado, la tira es inaccesible desde el portátil (y viceversa). Pruebas solo con el BT del teléfono apagado.
- **Una tira conectada NO se anuncia**: mientras está en conexión no aparece en escaneos. «No está en el aire» muchas veces significa «alguien retiene su enlace», no «murió». El diagnóstico de muerte solo tiene sentido sin nadie conectado.
- **PowerShell 5.1 no sirve**: los eventos WinRT (escaneo, notify) no se entregan (status=Started, packets=0); solo funcionan las escrituras. Todas las herramientas interactivas son C#.
- **Los modos dinámicos del mando** (arcoíris/chase) pisan los comandos de color estático — poner la tira en modo estático con el mando antes de las pruebas.
- **El checksum quizá no sea estricto**: una vez la tira pintó el color con checksum erróneo. Igual confiamos en la fórmula verificada.
- **El byte de secuencia importa**: un contador de paquetes repetido puede descartarse (de los docs DISCOVERIES) — incrementar siempre.
- **Registros ≠ nuestros comandos**: la trama de estado muestra registros que solo escribe el teléfono. Nuestros comandos cambian el color físicamente mientras los registros quedan viejos — la retroalimentación muestra que el enlace vive, no es espejo de nuestros envíos.

7. Caza de la «congelación» de la tira (2 nov, en curso)

> **Síntoma reportado por el usuario:** con la GUI a intervalo de 50 ms, tras 15–30 minutos la tira deja de actualizarse; la GUI reintenta la reconexión sin éxito; «la tira no está en el aire». Hace falta apagar y encender la alimentación. La luz NO se apaga — sigue pintando el último color aplicado.

Descartado: el propio ritmo de escritura

La reproducción directa falló: lednotify soak 50 ms × 24,4 min (24 000 comandos, mismo ritmo que la GUI) — **0 fallos**, enlace vivo, ritmo estable de 16,4/s; consulta posterior bien. El día anterior otros 3,5 min limpios. La carga constante de 20 Hz no mata al firmware. Detalle clave aportado por el usuario: la **parte LED sigue viva, la parte de radio muere** — los comandos no se reciben y la tira deja de anunciarse.

Hipótesis (por probabilidad)

#	Hipótesis	Razonamiento	Prueba
1	Punto muerto del firmware: un cuelgue raro del stack BLE desencadenado por la duración / patrones propios de la GUI (transiciones EMA, repeticiones) y no por el ritmo bruto	El soak envía un flujo de comandos extremadamente uniforme (barrido completo del arcoíris); la GUI genera un perfil real con muchas repeticiones de bytes idénticos	Pase de 1–2 h con la GUI real; el fichero de registro v1.5 marca el momento
2	Problema del stack/driver de Windows (enlace zombi)	«No está en el aire» se comprobó probablemente desde el portátil, cuyo enlace quedó obsoleto — la tira podía estar viva; el soak demuestra que el firmware es duro	En el momento de la congelación, abrir Surplife en el teléfono: si la ve → problema del PC; si no → firmware
3	Alimentación/térmica: caída de tensión del controlador bajo carga prolongada	Un reinicio de alimentación lo arregla — coherente	Reproducir a lo largo de días, apuntar si la descarga ayuda
4	Estado de la tira: un modo dinámico (arcoíris) pugna con los comandos estáticos	Quejas parecidas en los issues de la integración	Dejar la tira en modo estático con el mando

El kit v1.5 ya está listo para esto: el registro guarda sellos de tiempo en ms y el punto exacto del LINK LOST; la ventana automática avisa al usuario del desplome. Siguiente pase: GUI a 50 ms durante 1–2 horas, teléfono a mano para la comprobación en vivo en el instante de la congelación.

8. GitHub

README ×5docs/PROTOCOL.md §1–§4docs/AMBILIGHT.mddocs/BUILDING.mdsrc/*.csLICENCIA MITReleases v1.1–v1.4

Release	Tema	Activos
v1.1	Corrección de blanco cálido	AmbilightOT.exe
v1.2	Autoguardado + curva de saturación	AmbilightOT.exe
v1.3	Retroalimentación FF02 + brillo 0..100	lednotify.exe, ledctl.exe
v1.4	Modo rápido: 20 Hz verificado	AmbilightOT.exe, lednotify.exe
v1.5 (publicada)	Casilla de arranque + registro + ventana «LINK LOST»	AmbilightOT.exe, lednotify.exe + informes PDF ×5

Commits: `e820fcc` inicio → `16c6009` docs EN → `dc549a0` localizaciones → `52ea6c8` idiomas de la GUI → `7e90b94` blanco cálido → `6c5a10a` v1.2 → `3b9f305` MIT/capturas → `bf46696` v1.3 → `975371a` v1.4.

9. Backlog

Tarea	Estado	Notas
Lectura de estado (FF02)	**hecho (v1.3)**	query + decodificado + lednotify; límite documentado.
Escala de brillo 0..100	**hecho (v1.3)**	techo 100 confirmado.
Ritmo de 20 Hz	**hecho (v1.4)**	soak de 24,4 min limpio; «por encima de 17 Hz» sin sentido.
Arranque con Windows	**hecho (v1.5)**	casilla en la GUI + acceso directo en shell:startup.
Registro de fallos	**hecho (v1.5)**	fichero + botón + ventana «LINK LOST».
Diagnóstico de la congelación	**en curso**	pase GUI 1–2 h; comprobación por teléfono en el momento del desplome.
Selector de monitor (lista)	backlog	desplegable en vez de «principal / todas». Candidato v1.6.
Interpolación de transiciones	backlog	2–3 micro-frames de fundido entre colores en lugar de «teletransportes».
Tono 1° en vez de 2°	investigación	los grados impares se corrompen — probar primero.
Gamma de brillo	backlog	medios tonos más ricos en escenas oscuras.

10. Conclusiones

- **El protocolo está descifrado por completo** — desde la envoltura de transporte hasta los recovecos: familia v3, cuantización a grados pares, brillo en porcentaje, canal de consulta, trama de estado FF02, registros. Todo en PROTOCOL.md.
- **Ambilight es un producto, no un script**: GUI en 5 idiomas, autoguardado, reconexión automática, arranque con Windows, registro que se abre solo. Se compila sin Visual Studio.
- **Cada «límite» se levantó por experimento**: tres miedos propios despejados por medición — 16,9/s sostenido, techo 100, tope de 350 ms innecesario.
- **La regla «verificar con honestidad» dio sus frutos**: lo único abierto es una congelación rara que la carga sintética no reproduce; la caza continúa con la GUI real + un teléfono explorador en el instante del fallo.

Creado el 2 nov 2026 · Todas las cifras son mediciones reales de lednotify (rate/soak/ladder), no teoría · Fuente: Desktop\моды\лента\Project_Report*.html